Agent Skills

Write one, test it, fix it

Session M3-05 | 3 hours

Agenda

Segment	Time
What a skill is	10 min
Skill, agent, or MCP server?	15 min
Writing a good skill	10 min
Demo: `skill-creator`	12 min
Pros, cons, and tips	8 min
Lab handoff	5 min

What a skill is

A **folder** with a `SKILL.md` file. Copilot loads it when your request matches its description.

```
.github/skills/release-notes/
├── SKILL.md          # required: frontmatter + instructions
├── references/       # optional: read when needed
└── scripts/          # optional: run, not read
```

- **Project skills:** `.github/skills`, `.claude/skills`, `.agents/skills`
- **Personal skills:** `~/.copilot/skills`, `~/.agents/skills`

An open standard. Works in the CLI, the Copilot app, cloud agent, and IDE agent mode.

How Copilot uses one

1. You send a request.
2. Copilot compares it with each skill's **description**.
3. On a match, it loads `SKILL.md` into context.
4. It follows the instructions and can use files in the folder.

Or call one yourself: `Use the /release-notes skill for this.`

The description is the trigger. Get it wrong and nothing else matters.

Skill or something else?

Need	Use
Rule that always applies	Repository instructions
One saved prompt you run by hand	Prompt file
A workflow Copilot picks up when relevant	**Skill**
A persona with its own tools and scope	Custom agent
Live access to a system	MCP server

Rule of thumb: instructions are always on. Skills load on demand.

Skill vs custom agent vs MCP server

	Skill	Custom agent	MCP server
Gives Copilot	Know-how	A role	Reach
Is	Markdown, plus optional files	Markdown with tools and model	A running server
Loads	On demand	When picked or delegated	When configured
Risk	Wrong trigger	Too many tools	Credentials, data

They combine. An agent can use skills. A skill can teach Copilot to use an MCP server well.

Example: `grill-me`

A skill that **interviews you** about a plan.

- Reads your file first, so it doesn't ask what you already wrote.
- One question at a time, with a recommended answer.
- Stops when a colleague could build from the plan.
- Ends with decisions and open questions.

Not every skill makes a file. Some change how Copilot works with you.

More examples

Skill	What it does
`release-notes`	Writes changelog entries in your format.
`grill-me`	Interviews you before you plan.
`skill-creator`	Helps you write and test skills.
`pdf`, `docx`, `xlsx`	Bundle scripts that read and write files.

Frontmatter

```
---
name: release-notes
description: Write release notes or a CHANGELOG entry from merged PRs.
  Use when the user asks for release notes, a changelog entry, or
  "what shipped". Don't use for commit messages or PR descriptions.
---
```

- `name`: lowercase with hyphens, same as the directory
- `description`: **what it does and when to use it**
- Optional: `allowed-tools`, `argument-hint`, `user-invocable`, `disable-model-invocation`, `license`. Check the current docs

Write the description

Weak: `Helps with releases.`

Better: says what, when, and when not.

- List the words people actually use.
- Add a "Don't use for…" line next to look-alike tasks.
- Be a little pushy. Skills tend to undertrigger.

Write the body

- Use **imperative steps**: "Read the top two entries."
- Explain **why** a rule matters. The model generalizes better from reasons than from shouting.
- Show a short example of the output.
- Say what to do when input is missing: ask.
- Keep `SKILL.md` short. Move long templates to `references/`.

Bundle files

```
SKILL.md        → "Group PRs using references/format.md"
references/    → loaded only when the step needs it
scripts/        → run for deterministic work
```

Scripts need `allowed-tools` to run without a prompt. **Don't pre-approve `shell`** unless you've read every script and trust the source.

Demo: `skill-creator`

`skill-creator`

A skill that helps you write skills.

1. **Interviews you:** purpose, triggers, output, edge cases.
2. **Drafts** `SKILL.md` and bundled files.
3. **Suggests test prompts.**
4. **Revises** from what you saw.

Its eval and description-optimizer scripts call the `claude` CLI. Skip them. Test by hand.

Pros

- **Repeatable:** same workflow every time.
- **On demand:** loaded only when relevant, unlike always-on instructions.
- **Shareable:** commit it, review it, version it.
- **Portable:** an open standard across Copilot surfaces.
- **Bundles files:** templates, references, scripts.

Cons

- **Triggering is probabilistic.** It can miss or misfire.
- **Failures are quiet.** No error when it doesn't load.
- **Skills collide.** Overlapping descriptions confuse selection.
- **They go stale** like any doc.
- **Third-party skills are code.** Scripts can run.
- **Support differs by surface.** Check yours.

Tips

- Start from a real task you've done by hand.
- Test three prompts: should trigger, shouldn't, ambiguous.
- Run each twice. Variance is normal.
- Fix one thing at a time, then rerun.
- If it's one always-on rule, write instructions instead.
- Delete skills nobody uses.

Lab handoff

1. Run a baseline with no skill.
2. Install `skill-creator`.
3. Create `release-notes` with it.
4. Test three prompts live.
5. Fix what failed.
6. Build `grill-me` and interview yourself about `idea.md`.
7. Decide: skill, custom agent, or MCP server?
8. Review one third-party skill.

Submit your test results, both skills, and `decisions.md`.